Concurrent Hash Tables - CMSC 23000

Joshua Miller

March 12, 2014

1 General Design

Modules

- util.h – contains macros and definitions for error handling and debugging
- hasht.h, libhasht.a – contains definitions for each of the four hash table types
- tests.c – contains the tests for the entire package
- app.h – contains the different hash performance tests Serial, Parallel, ParallelNoLoad.
- experiment{1,2,3,4}.c – contains the code to run the experiments.

2 General

The first comment I would like to make about this project is that the rates I gathered during the experiments varied an extereme amount.

The hash table created for this project are contained in the statically linked library `libhasht.a` located in `src/libhasht` with `hasht` being the prefix for all cusom hash tables. The general design of the hash tables if a polymorphic inheritance from the main struct hasht_t. This 'object' contains pointers to it's initialization, add, remove, resize, and contains functions. Included below is the majority of hasht.h. I apologize for the inclusion a nearly complete header file, but I feel that the information included in it is an encompassing description of the hasht abstraction. Each 'subclass' is described in later sections.

```
/* The top level hash table abstraction */
typedef struct hasht_t hasht_t;

/* Pointer types for each table interaction member */
typedef int (*hasht_init_f)    (hasht_t *, int, int);
typedef int (*hasht_free_f)    (hasht_t *);
typedef int (*hasht_get_type_f)(hasht_t *);
typedef int (*hasht_add_f)     (hasht_t *, void*, int);
typedef int (*hasht_contains_f)(hasht_t *, int);
typedef void *(*hasht_remove_f)  (hasht_t *, int);
typedef void *(*hasht_resize_f)  (hasht_t *);

/* Enumeration to be contained in hasht object */
typedef enum hasht_type_t
{
    LOCKING,
    LOCKFREEC,
    LINEAR,
```

```
    AWESOME
} hasht_type_t;

typedef struct hasht_locking_t
{
    ll_t **buckets;
    pthread_rwlock_t *rwlocks;
    int n_rwlocks;
} hasht_locking_t;

typedef struct hasht_lockfreec_t
{
    ll_t **buckets;
    pthread_rwlock_t *rwlocks;
    int n_rwlocks;
} hasht_lockfreec_t;

typedef struct linear_node_t{
    unsigned int key;
    unsigned char inuse;
    void *data;
    unsigned int steps;
} linear_node_t;

typedef struct hasht_linear_t
{
    int n_locks;
    int resizing;
    pthread_mutex_t *locks;
    linear_node_t *buckets;
} hasht_linear_t;

typedef struct awesome_node_t awesome_node_t;

struct awesome_node_t{
    unsigned int residue;
    unsigned int key;
    void *data;
    awesome_node_t *child;
    unsigned char inuse;
    unsigned char valid;
};

typedef struct hasht_awesome_t
{
    awesome_node_t root;
 } hasht_awesome_t;

struct hasht_t
{
    /* Type specifier */
    hasht_type_t type;

    /* Scalar properties */
    int logsize;
    int capacity;
    int count;

    /* Member Function Pointers */
    hasht_add_f      add;
    hasht_contains_f contains;
    hasht_remove_f   remove;
    hasht_resize_f   resize;

    /* Member types */
```

```
    hasht_locking_t      locking;
    hasht_lockfreec_t    lockfreec;
    hasht_linear_t       linear;
    hasht_awesome_t      awesome;
};


/* Constructor for new hash table */
hasht_t *hasht_new(hasht_type_t type, int capacity, int expected_threads);
int hasht_locking_init(hasht_t *table, int capacity, int expected_threads);
```

To avoid the cost of freeing memory during computation. I wrote a rudimentary garbage collector. During computation, allocations are stored in a reference array. After computation a call to GARBAGECOLLECT() frees all of the memory allocated by the hash tables.

3 Lock-based Closed-address

The HASHT_LOCKING_T table contains the following

- all inherited properties from HASHT_T
- an array of linked lists of size MAX_DEPTH
- an array of read write locks of size NLOCKS
- an integer COUNT specifying how many packets are in the table

Because we can stripe the hash table, the locks are held in parallel array, such that the lock for item i is accessed by calculating $i \mod$ NLOCKS.

Editing (writing, resizing) the hash table requires the effective acquisition of both the write and read locks. Reading (contains) from the hash table requires acquisition of only the read locks. This means that for reads and writes will interfere with each other, this being the largest source of contention and overhead. The difficulties in implementing this were slightly more challenging than implementing the LOCKFREEC because the later consisted more of modifications than original design. The resize on the LOCKING table is accomplished by acquiring all the locks, creating a new and duplicate bucket array, then redirecting the table's bucket array to the newly created one. This is all sorts of painful because (1) we have to lock the entire table and (2) we have to go through the trouble of re-adding each element to the new bucket array. Avoiding these two factors would eventually become part of the design requirement for my AWESOME table. Furthermore, it was important to verify that the table had not been resized while acquiring the write lock for an add or remove call. If it had been, then the respective write call would simply be recusrively called again after freeing any obtained locks. A resize call that acquired the locks during an second extern resize would simply return.

4 Lock-Free-Contains Closed-Address

The HASHT_LOCKFREEC_T table contains the following

- all inherited properties from HASHT_T
- an array of linked lists of size MAX_DEPTH
- an array of PTHREAD_MUTEX_T of size NLOCKS
- an integer COUNT specifying how many packets are in the table

Again, the locks are held in parallel array, such that the lock for item i is accessed by calculating i mod NLOCKS.

The major design changes from the HASHT_LOCKING_T were (1) checking the validity of the table before returning from a contains call and (2) making sure that nodes were not deleted while a contains was attempinting to traverse a list. The later objective was easily obtained because my implementation of the locking table lent itself an atomic replacement of the entire bucket list. The former objective was acheived simply by maintaining a record of the table size at the beginning of the call to contains and comparing before exiting. If this test fails, then the contains method is simply called again.

5 Linearly Probed Open-Address

The table contains the following

- a single array for nodes each node containing
  - a pointer to the data
  - a key
  - a binary setting specifying availability
  - a step count
- a size

Finding an element in the linear probe hash table involved masking the key, and selecting the respecitve node from the array. After a call to ADD() acquiring the lock (again a striped locking scheme) if the node is available then modify the availibility flag to signify that it is now taken. If we have to search along the array, then we note in the original location how far we had to travel, or the maximum if another call has already noted its travel distance. I made the silly mistake of setting this counter to 0 on a REMOVE() call, which caused CONTAINS() to return incorrectly.

When a resize happens, the locks for the entire array are acquired and a new array of nodes are created. The old items are then re-added to the new array before the locks are released.

6 The most awesomest hash table that concurrent data structures has ever seen, a.k.a AWESOME

The two main focal points that I wanted to improve were (1) locking and (2) resizing (3) locality. All of the above designs required a stop-the-world resizing strategy, and I wanted to create a design that was free of this long pause progress. Furthermore, I wanted to maximize the spacial locality of my data structure; I decided to use arrays, rather than linked lists, as the backbone for this hash table. The problem with arrays, however, as we saw with the linear probe is the resize. For these reasons I came up with the following design. The idea is a dynamicly multidimensional hash table somewhere between a linear probe and a tree.

Consider the table to be one dimensional at initialization. Add calls proceed as if the table were a linear probe hash table, except each call is only allowed to walk down the array a number of times specified by the OPTIMISM parameter. This parameter specifies how optimisitic there will be space in the current dimension. If an open position is found within this limit, the item is inserted as in a linear probe. However, if there is no longer room within the limit of the initial hash position, a new dimension is added. This dimension branches off of the node itself. In this way it is like a tree, but

each branch is not a single item but a 'dimension' with a preset length DIM_LEN. Therefore, each dimension consists of an array of nodes, which may or may not have dimensions branching off of them. If the algorithm is unable to find a desired location/item in the current dimension, then the algorithm steps into the next dimension. Because each item in a new dimension will have the same index in the old dimension, a new array index is calculated by bit shifting the key to the right by the dimension level and remasking on DIM_LEN.

Through the use of atomic operations and recursively incremental resizes, I was able to make the table completely lock free. There is no resize call, an add call simply inserts a dimension if necessary. N.B: The new dimension must be attached with an atomic compare and set call, which prevents replacing a dimension allocated and attached by a concurrent ADD() call.

The trade off of this table is (a) locality, a lock-free nature, and quicker tree-like lookup versus (b) pre-allocating node arrays which may be in excess. Therefore I ran an extra experiment to optimize the parameters OPTIMISM and DIM_LEN. A plot of the results are presented below. The results from this experiment were used as settings for the AWESOME lock in successive experiments.

Moreover, because of the tree-like nature and the shifted address masking, if any given key is contained in the table, we know exactly which nodes it had to pass through at each dimension of the table. Therefore, I kept a RESIDUE on each node. As an ADD() call either passed through a node to a new dimension, or assigned an item to the node, it atomically OR-ed the key onto the residue. In this way, a contains or remove call knows in advance if a key is absolutely *not* present in recursive levels of the table. This however, does not help a lookup if the key has been added and successively removed from the table.

7 Tests

Each test follows the same testing scheme through progressively more rigorous composite tests. First the add and remove calls are tested by verifying containment for small numbers of elements. Next, a battery of random keys are added to the table. Then a similar scheme but with more than one thread. The final test walks over a key space in a pseudo-random cycle 4 times. Each key is added, with a random possibility of being removed afterwards. The testing function keeps track of and tests against containment in an array parallel to the key array. I am very satisfied with this test suite.

To run the tests

```
make tests
./test
```

Or to test a specific table

```
./test LOCKING
./test LOCKFREEC
./test LINEAR
./test AWESOME
```

Expected output:

```
TEST: [                 add/remove] [ serial_simple_hash_test(AWESOME)] [ PASSED ]
TEST: [          add/remove/resize] [ serial_resize_hash_test(AWESOME)] [ PASSED ]
TEST: [         and/resize/contain] [     parallel_hash_test1(AWESOME)] [ PASSED ]
TEST: [  random traversal of key..] [     parallel_hash_test2(AWESOME)] [ PASSED ]
```

8 Experiments

8.1 Dispatcher Rate

This experiment involves running the parallel test with no load and minimal packet size for the number of threads equal to the number of cores. This will present us with the rate at which the dispatcher thread can enqueue the packets onto the lamport queues. I predict that the dispatcher rate will be based upon the relative performanc of the HASHGENERATOR, which I believe will be slightly slower, and therefore the performance will fall just below the performance of the dispatcher rate in previous projects.

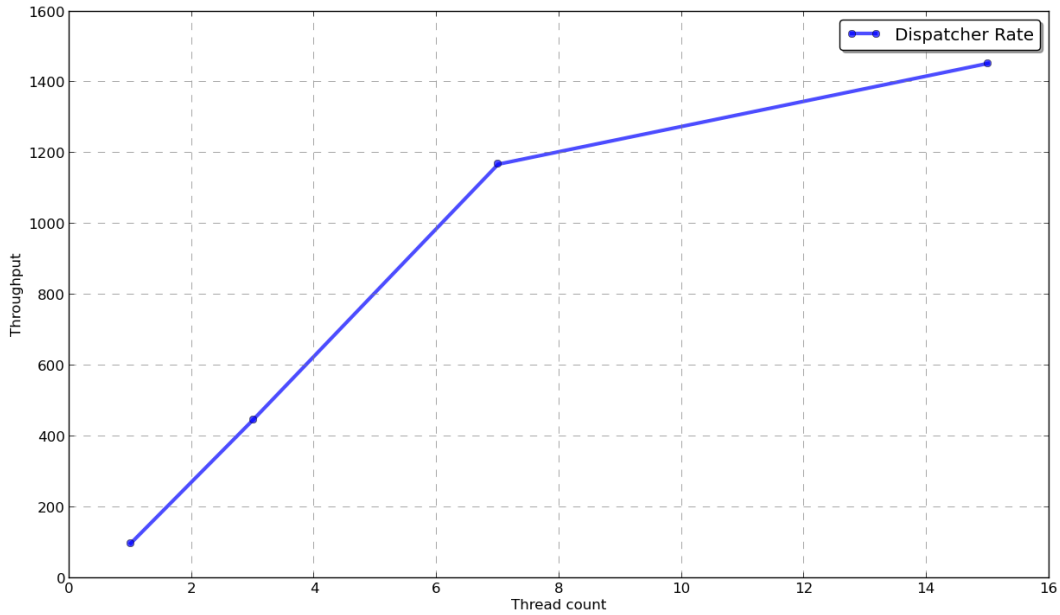

Figure 1: Dispatcher rate as a function of thread count

8.2 Parallel Overhead

Configuration: $n = 1$, $W = 4000$

This experiment involved running the parallel test with each of the parallel hash tables, but on a single thread. In comparison with the performance of the serial hash table the speedup will be less than 1.

Hypothesis: I predicted that for all hash tables, the speedup will be lower for the heavy write runs. This is because the writing has an added complexity relative to the serial version to handle concurrency that the reading does not.

Table 1: Mostly Reads

Table Type	Rate	Speedup
SERIAL	-	-
LOCKING	59.57	
LOCKFREEC	51.47	
LINEAR	57.99	
AWESOME	53.92	

Table 2: Heavy Writes

Table Type	Rate	Speedup
Serial	-	-
Locking	55.64	
LockFreeC	80.21	
Linear	51.85	
Awesome	52.09	

Result:

8.3 Speedup

Configuration: $n \in \{1, 2, 4, 8, 16, 32\}$, $W = 4000$ with varying load schemes.

This experiment involved running a scaling test on all the parallel hash packet implementations.

Hypothesis: I predicted that our program will be able to acheive 80% of ideal speedup, or just over 12 times the serial version on 16 cores.

8.4 Mostly Reads

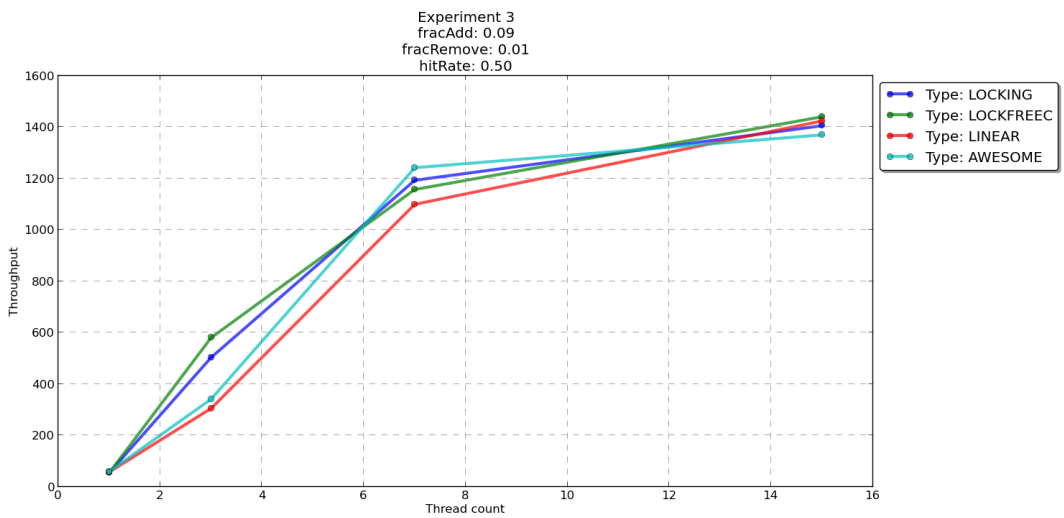

Figure 2: Table speedup comparision $P_+ = .09$ $P_- = .01$ $\rho = .5$

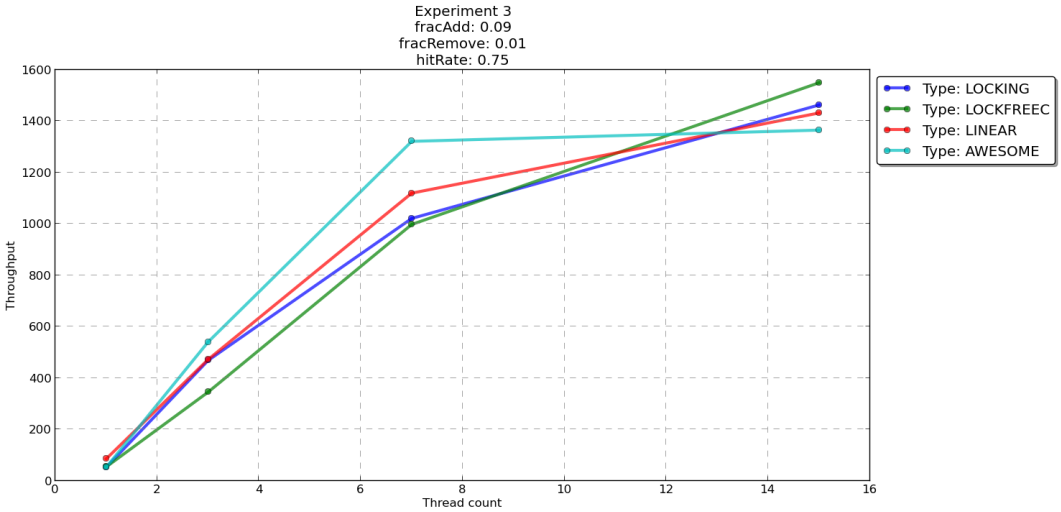

Figure 3: Table speedup comparision $P_+ = .09$ $P_- = .01$ $\rho = .75$

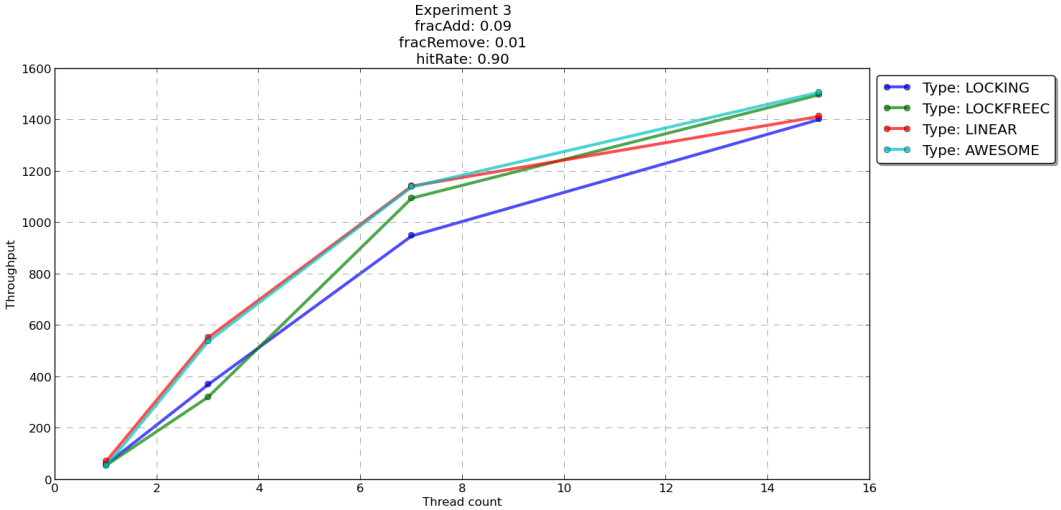

Figure 4: Table speedup comparision $P_+ = .09$ $P_- = .01$ $\rho = .9$

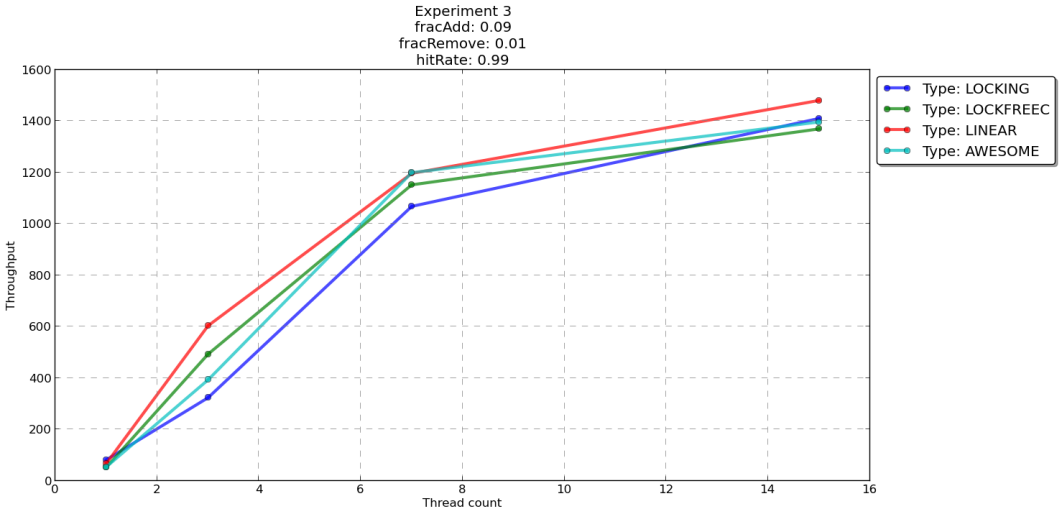

Figure 5: Table speedup comparision $P_+ = .09$ $P_- = .01$ $\rho = .99$

8.5 Heavy Writes